

Linear#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.linear.Linear.html>

Description:

Applies an affine linear transformation to the incoming data: $y = xA^T + by = xA^T + b$. This module supports TensorFloat32. On certain ROCm devices, when using float16 inputs this module will use different precision for backward. `in_features` (int) – size of each input sample
`out_features` (int) – size of each output sample

Function Signature

```
class torch.nn.modules.linear.Linear(in_features,  
out_features, bias=True, device=None, dtype=None)[source]#
```

Parameters

Shape:

Variables

```
extra_repr()[source]#
```

Returns:

Tensor

Code Examples

```
>>> m = nn.Linear(20, 30)
>>> input = torch.randn(128, 20)
>>> output = m(input)
>>> print(output.size())
torch.Size([128, 30])
```

Detailed Documentation

Documentation

Applies an affine linear transformation to the incoming data: $y = xA^T + b$ by $y = xA^T + b$.

This module supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

`in_features` (int) – size of each input sample

`out_features` (int) – size of each output sample

bias (bool) – If set to False, the layer will not learn an additive bias. Default: True

Input: $(*, H_{\text{in}})(*, H_{\text{in}})$ where $*$ means any number of dimensions including none and $H_{\text{in}} = \text{in_features}$

Output: $(*, H_{\text{out}})(*, H_{\text{out}})$ where all but the last dimension are the same shape as the input and $H_{\text{out}} = \text{out_features}$

weight (torch.Tensor) – the learnable weights of the module of shape $(\text{out_features}, \text{in_features})$. The values are initialized from $U(-k, k)$, where $k = \frac{1}{\sqrt{\text{in_features}}}$

bias – the learnable bias of the module of shape (out_features) . If bias is True, the values are initialized from $U(-k, k)$, where $k = \frac{1}{\sqrt{\text{in_features}}}$

Return the extra representation of the module.

Runs the forward pass.

Resets parameters based on their initialization used in `__init__`.